FRED TALKS

2nd November 2026

CONTENTS

What is Codemode

ARMIN RONACHER · 2870 WORDS

Test Coverage Won't Save You

JENN COOPER · 1991 WORDS

What is Codemode

ARMIN RONACHER · 06 OCT 2026 · SOURCE

More than a year ago I wrote a few posts here that recommended people not to load custom tools into their context (or MCP servers) but to just use more scripts. Most importantly I wrote that Code Is All You Need and I wrote about that MCP needs code. With Pi 1.0 we now added MCP support via Codemode which in some ways is a long time coming, but then also maybe somewhat surprising to some. So I want to share some updated thoughts on this blog on what this all means.

What Are Tools

When a harness like Pi provides tools for an LLM to call, it does so by supplying some tool definitions which then translate into some token structure on the server side. Whether a model is encouraged to call a tool is the result of the reinforcement learning process. Something I wrote about before if you want to learn more.

One of the reasons we strongly lean towards CLI and bash is because it allows easy composition of calls, and because the model also learns how the file system works when it's trained. So when it invokes a tool like `echo foo > /tmp/test.txt` the model also learns that after that tool call, there is now a file called `test.txt` in `/tmp`.

However bash has one fundamental limitation which is that it can only compose programs that run. And there are some things, which are not programs, but native tools to the LLM and they sort of have to be.

The most obvious example here is `read` or `view_image`. If a multimodal model needs to read an image, it cannot use `cat` for that because the harness needs to inject the actual image payload into the protocol of the LLM.

Another quite vivid example are sub agents. In order to spawn and orchestrate sub agents, it's tricky to avoid tools that are provided by the harness. While in theory the agent could provide a CLI tool that talks to the outer harness via environment variables and Unix sockets, it's a rather crude process. It however has another issue, and that is where the code runs.

Brains vs Hands

To better understand that, it's important to think a bit more about where all the bits and pieces run. There really usually are two different systems involved. The first is the brain, the harness: it runs on one machine. It's trusted. The second is *often* the same machine, but it's really where the tools are executing: the hands. In Pi we now call this the execution environment, but you can think of it as the target of all the operations.

Crucially what is important for us, is that there is a dividing line between the harness brain and the target environment that runs bash and executes the tools.

And splitting this in half has some really important consequences. For a start it means that they are running on different file systems and they have different levels of trust. If you for instance use a sandboxing solution like Gondolin your bash stuff will be sandboxed just fine, but the harness itself will not be.

Orchestrating The Harness

Which brings us to what Codemode really does: it's a way for the LLM to express and orchestrate complex operations on the harness side, but not the execution environment side. Codemode runs in the harness, in its own sandbox. In case of Pi it's running in QuickJS within a WASM runtime with intentional limitations: no network, no file system, no timers, limited RAM. The only way is to call more tools. You could also imagine that Codemode could run Scheme or some other language as well.

If you are not familiar with Codemode, it's basically just a way to issue tool calls from within some language, in our case JavaScript. That allows you to compose those calls without necessarily going through the LLM's context. Credit for naming goes to our friends at Cloudflare who coined it.

For instance if you issue a bash call as a regular tool call in the LLM, then we only throw the trailing 2000 lines into the context and if the agent wants more, it needs to look at the overflow file itself. If however the agent issues that invocation via Codemode, then the Codemode side gets larger outputs sent structurally.

Most importantly, because Codemode is JavaScript the agent can express concurrent operations and basic workflows. A common way in which you see agents now use this, is to first probe at 5-10 items from some tool response to see what it looks like, and to then write a Codemode script that processes the next n items.

Codemode also allows you to throw state into the transcript! That means that one Codemode invocation can stash away data, that the next call in the session can load again. And remember: this is on the harness host, not the sandbox.

In case of Pi, Codemode also allows you to issue calls that naturally do not make any sense in Pi's traditional interface. For instance if you want to generate images with an image model or you want to classify some text with a one shot classifier model, those Pi APIs are exposed via Codemode, but not via regular tools where they would just waste context.

What It Looks Like

So now that we talked a bunch about it, it's probably worth being a bit more explicit about it. Let's walk ourselves through some invocations of Codemode of recent Pi sessions of mine. Note that none of this code is human written. It's from real sessions of Pi, just re-indented for your viewing pleasure. The agent starts using Codemode automatically either because it's a task where the model already naturally picks up that tool, or because a user asked it to.

Note that Codemode is by default only enabled in Pi when MCP is enabled, but you can turn it on with `"defaultTools": ["+codemode"]` in the settings. Just ask Pi to enable it for you.

Generating Images

Let's start simple with image generation. Image generation is a feature that Pi supports in the AI SDK core, but it's not a tool that the agent can use. In the past the only way to use image models has been to write a bespoke extension or to have the agent run node itself and use the internal image APIs. However because we expose quite a few of the internal model APIs within Codemode, it means that the agent can use it:

```
const [painter] = await models.getAvailableOfType("image");
const result = await models.generateImages(painter, {
  input: [{ type: "text", text: "A cute little puppy sitting on a gra
    "lawn, soft natural light, photorealistic" }],
});
if (result.stopReason !== "stop") return result.errorMessage;

for (const block of result.output) {
  if (block.type === "image") image(block);
```

```
  else text(block.text);
}
```

Note that the call to `image()` sends the image back as image content to the LLM. On the harness side it feeds it directly into both the agent, as well as onto disk as a temporary artifact in case the agent wants to be able to pass that image back to bash.

Classifying Things

Similar things apply to classifier models such as Jev. They also do not fit well into the workflows of an agent through the typical tools. But rather than making a bespoke tool available, Codemode just allows the agent to reach into the AI SDK and invoke those directly. Here you can see how Jev is used to mass process GitHub issues for a quick sentiment analysis:

```
const jev = await models.getModelOfType("classifier", "typesafe", "je
const r = await tools.bash({
  command: "gh issue list --state open --limit 100 " +
    "--json number,title,body,comments",
});
const issues = JSON.parse(r.output);

const results = await Promise.all(issues.map(async (issue) => {
  const res = await models.classify(jev, {
    state: {
      title: issue.title,
      body: (issue.body || "").slice(0, 4000),
      comments: issue.comments.slice(-5).map(c => c.body.slice(0, 800
    },
    questions: {
      sentiment: {
        type: "choice",
        instructions: "What is the overall sentiment of the author to
        criteria: {
          positive: "Appreciative, happy, constructive praise",
          neutral: "Matter-of-fact report or request without emotion"
          negative: "Frustrated, annoyed, upset, or angry",
        },
      },
      frustration: {
        type: "score",
```

```
        instructions: "How frustrated is the reporter?",
        criteria: ["not at all", "mildly", "clearly frustrated", "ver
      },
      kind: {
        type: "choice",
        instructions: "What kind of issue is this?",
        criteria: {
          bug: "Bug report or regression",
          feature: "Feature request or enhancement",
          question: "Question or support request",
          other: "Docs, discussion, meta, spam",
        },
      },
    },
  });
  if (res.stopReason !== "stop") {
    return { n: issue.number, title: issue.title, error: res.errorMes
  }
  return { n: issue.number, title: issue.title, ...res.answers };
}));

store("sentiment_results", results);
return results
  .filter(r => !r.error)
  .sort((a, b) => b.frustration.score - a.frustration.score)
  .slice(0, 12)
  .map(r => `#${r.n} ${r.frustration.score.toFixed(2)} [${r.kind.choi
```

Note how in that above example we also call `store()` which dumps the result of that execution into the session transcript. A future invocation of Codemode can thus read back that result if it wants to.

The `Promise.all` here is fine, because Pi limits the total number of concurrent tool executions itself to four and maintains a queue for the rest.

A more adventurous example is to use Jev to drive a game engine for debugging purposes:

Codemode with Jev for Game Debugging

Here it knows about my `tankctl` command and it built itself quickly a minimal harness around it to drive a game loop to assist a user with debugging a problem. Note how it built a 30 step loop in which each step goes back to both the game engine to get a text dump of what's going on, and then to Jev to determine what to do next:

```
const jev = await models.getModelOfType("classifier", "typesafe", "je
const tank = async (cmd) =>
  (await tools.bash({ command: `tools/tankctl "${cmd}"` })).output;
await tank("start --map assets/maps/night_arena.map");

const questions = {
  action: {
    type: "choice",
    instructions: "You control the tank '@' in a top-down tank game.
      "Choose the best next action.",
    criteria: {
      attack: "an enemy has line of sight to you and you can fire at
      approach: "no enemy has line of sight; drive toward the nearest
      dodge: "an enemy shot is heading at you and will hit soon",
      powerup: "a powerup is close and no enemy threatens you",
    },
  },
};

function commandFor(choice, st) {
  const p = st.player;
  const enemy = st.enemies.filter(e => !e.dead)
    .sort((a, b) => (b.los - a.los) || (a.dist - b.dist))[0];
  if (choice === "attack" && enemy) {
    return `fire_at tank ${enemy.id}; frames 30 until clear,damage,ki
  }
  if (choice === "dodge") {
    // move perpendicular to the closest incoming shot
    const s = st.projectiles.filter(s => !s.yours)
      .sort((a, b) => a.eta - b.eta)[0];
    const dir = s && Math.abs(s.vel[0]) > Math.abs(s.vel[1])
      ? (p.pos[1] > s.pos[1] ? "+down" : "+up")
      : (p.pos[0] > (s ? s.pos[0] : 0) ? "+right" : "+left");
    return `input ${dir}; frames 20 until damage; input stop`;
```

```
  }
  const powerup = st.powerups.filter(u => u.available)
    .sort((a, b) => a.dist - b.dist)[0];
  if (choice === "powerup" && powerup) {
    return `goto ${powerup.pos[0]} ${powerup.pos[1]} 180`;
  }
  return enemy ? `goto ${enemy.pos[0]} ${enemy.pos[1]} 90` : null;
}

const log = [];
for (let step = 0; step < 30; step++) {
  const st = JSON.parse(await tank("state"));
  if (st.state !== "playing") break;
  const threats = st.projectiles
    .filter(s => !s.yours && s.miss_dist < 1.5 && s.eta < 1.5)
    .map(s => `incoming shot dist ${s.dist} eta ${s.eta}s`)
    .join("\n") || "no incoming shots";
  const r = await models.classify(jev, {
    state: { map: await tank("view 8"), threats, hp: st.player.hp },
    questions,
  });
  if (r.stopReason !== "stop") {
    log.push(`#${step} classifier error: ${r.errorMessage}`);
    break;
  }
  const choice = r.answers.action.choice;
  const cmd = commandFor(choice, st);
  if (!cmd) break;
  log.push(`#${step} hp=${st.player.hp} ${choice} -> ${await tank(cmd
}
return log.join("\n");
```

Calling MCP Servers

Lastly, Codemode obviously is great for calling MCP servers. And because we do not actually expose any of the MCP tools to the LLM, the agent first uses provided APIs to issue a tool search within Codemode to discover what it might be able to do with the connected servers. This form of progressive discovery makes the whole MCP business work well enough for a lot of use cases today.

Here for instance you can see the agent reach for the Sentry MCP straight away, even without discovering the tools, presumably because it has learned during the RL process already about what the Sentry MCP looks like. But it learns from what we inject into the system prompt, that the Sentry server is available to begin with. It's not completely guessing here.

```
const orgs = await tools.mcp__sentry__find_organizations({});
const { organizations } = orgs.structuredContent;
const results = await Promise.allSettled(organizations.map(org =>
  tools.mcp__sentry__find_projects({
    organizationSlug: org.slug,
    regionUrl: org.regionUrl,
  })
));
return organizations.map((org, i) => {
  const r = results[i];
  if (r.status !== "fulfilled") return { org: org.slug, error: String
  if (r.value.isError) return { org: org.slug, error: r.value.content
  return {
    org: org.slug,
    projects: r.value.structuredContent.projects.map(p => p.slug),
  };
});
```

Modern MCP Is A Fight

I really don't want to talk too much about MCP here, but MCP is in fact a protocol that greatly benefits from Codemode. The problem in parts is that MCP in practice often targets harnesses that do not (yet?) use Codemode. But the tide is shifting. In the meantime, a temporary crutch has been to do what Cloudflare did, and do Codemode within the MCP server. But now we have Codemode in Codemode which is pretty bad. It means double JSON escaping, easy for smaller models to get confused by and the inner code cannot call the outer tools. So if you for instance use the Cloudflare MCP servers in Pi, the agent needs to write JavaScript and funnel it through more JavaScript. This is really not optimal, but it's also understandable that this is happening:

```
const accRes = await tools.mcp__cloudflare__execute({
  code: `async () => {
    const r = await cloudflare.request({ method: "GET", path: "/accou
    return r.result.map(a => ({ id: a.id, name: a.name }));
  }`,
```

```
});
const accounts = JSON.parse(accRes.content.map(c => c.text).join(""))

const out = [];
for (const account of accounts) {
  const r = await tools.mcp__cloudflare__execute({
    account_id: account.id,
    code: `async () => {
      const r = await cloudflare.request({
        method: "GET",
        path: \`/accounts/\${accountId}/workers/scripts\`,
      });
      return r.result.map(s => ({ id: s.id, modified: s.modified_on }
    }`,
  });
  out.push({ account: account.name, workers: r.content.map(c => c.tex
}
return out;
```

MCP Desires

So to end things off: how well does Codemode work with MCP today? Well … not amazingly well. That's because MCP servers are not really targeting harnesses that use Codemode yet (though at this point I think most harnesses support it).

For this to work well some recommendations:

- **Structured content:** Codemode wants calls to return some nicely formatted JSON. So that needs to come back from the server, and many don't do that yet. The `outputSchema` system in MCP is great for that.

- **Consistent results:** an interesting failure case is when an MCP server does not return consistent data. For instance because it tries to token optimize things depending on how many items are in the result set. This can cause an initial probe with 5 items to succeed, but then fail when the server returns the maximum batch size.

- **Large binary data:** today MCP does not yet support large binary data so quite a few use cases that are really interesting do not work well at all yet. You end up with all kinds of weird workarounds such as pre-signed URLs to allow file uploads then to happen through non MCP channels.

- **Composable tool search:** the MCP server might know better than the MCP client which tool is appropriate for a task. But there is no good mechanism today that allows a harness to fan out tool searches across multiple MCP servers. It's all emergent behavior and it does not scale well to multiple active servers.

Future of Codemode

So where does this leave us? Is this a reversal of what I wrote a year ago where I encouraged CLIs? I don't think so. In fact, the MCP ecosystem from my perspective picked up on exactly what we pointed out a year ago works: code. But Codemode goes beyond MCP in that it can act as a capable mechanism within the harness to express more freedom for the agent.

There are however also some things that we still need to figure out. For one, durability with Codemode is trickier. We might have to adopt some ideas from durable workflow engines here to snapshot invocations. Or maybe, something like Starlark is a better composition language than JavaScript given its deterministic nature.

Images, binary data and just the inability of this pattern to work with smaller models is also something that needs to be fleshed out. So it's for sure not a perfect solution yet, but it's quite a useful pattern that I expect us to leverage more.

Test Coverage Won't Save You

JENN COOPER · 09 JUN 2026 · SOURCE

A year ago, if a codebase accumulated three different ways to do the same thing, somebody would usually notice.

A reviewer might leave a comment. A senior engineer would mutter something about "yet another helper," and eventually the team would clean it up. Maybe they'd write a short doc. People would learn, collectively, that this was not the way.

This was not a perfect system. Human review can be slow. People miss things. Pattern docs get stale. Engineers develop opinions that are 70% wisdom and 30% scar tissue.

Still, the friction did something useful. It kept codebases from drifting too quickly.

Coding agents change that.

> A room with too much electricity.
>
> "Now, this is a room with electricity. But it has too much electricity."

Once agents are writing a meaningful share of code, the question is no longer just "can we get working code faster?" Obviously, yes. We can. Sometimes hilariously so.

The more interesting question is: what does the codebase teach the next agent?

Because it does teach.

Every merged PR becomes a precedent. If the repository contains one clear pattern, the next agent has a decent shot at following it. If it contains three slightly different patterns, the next agent may extend one, combine two, or invent a fourth for reasons that are, in technical terms, vibes.

The funhouse builds itself

Coding agents, like some of us, have a chaotic inner goth teenager.

Their non-deterministic nature means they can be inconsistent, often in surprising ways. They can generate the weird little leap that solves the problem, or they can generate novelty where nobody asked for it.

The dangerous thing is that most of the code is fine, if you look at it in isolation.

A PR passes tests. The implementation makes sense. The file is clean enough. You look at the diff and think, yeah, sure, that seems okay.

But zoom out, and the codebase as a whole has started to get weird.

- three near-duplicate utilities
- multiple data-fetching patterns
- parallel abstractions solving almost the same problem

The codebase remains locally **cohesive**, while slowly losing global **coherence**.

That matters more than it used to, because even more than humans, agents are responsive to the signals around them. A clear codebase gives them clear precedent. A muddled codebase gives them confusion.

You can put principles in `CLAUDE.md` or docs, but they're... advisory. They compete with task instructions and inline comments for attention, get summarized when context windows compact, and depend on the agent attending to the right instruction at the right moment.

If your codebase contains three slightly different ways to solve a problem, the next agent has to infer which one is canonical.

From the agent's point of view, this is not irrational. The repository gave it mixed evidence.

The scary outcome is not code that obviously fails. It is code that superficially keeps working, while the shape of the system increasingly gets worse.

Tests are nice

Most discussions about AI-native development jump from this problem – agents' tendency to accumulate tech debt – directly to tests.

And yes, across the industry, teams are writing dramatically more tests than they ever have. Agents have made high test coverage affordable in a way it never used to be.

Recently, Garry Tan argued that the primary way to keep AI agents on track is 90% test coverage:

> *Tests are the ratchet. 90% coverage, every PR, no exceptions.*

And indeed, test coverage is the simplest kind of ratchet: a mechanism that allows motion in one direction only, like a socket wrench that turns the bolt forward but never lets it spin back. Once a test locks in a behaviour, it becomes difficult to accidentally regress.

But it's worth being specific about what tests actually do.

Unit tests check that a function still returns what it returned before. Integration tests check that pieces still wire together the same way. E2E tests reach further: they check whether the product still does what it used to, and the assertions you prioritize there should be an important act of human judgment.

But they all share the same basic shape: tests verify that code does what it did before.

Whether what it did was even the *right way to do it* is a separate question.

Tests ratchet behavioural sameness.

Meanwhile docs and evals can pin down reasoning and behaviour bars, which is sometimes more important. But none of these is directly looking at the *shape* of the system.

90% coverage of good patterns

A coverage ratchet only improves the codebase if the patterns it's locking in are already good ones.

If a questionable pattern already exists in the code, the next agent is more likely to extend it. The tests generated around that implementation only reinforce it further.

Over time, high coverage can unintentionally fossilize architectural drift, rather than prevent it in the first place.

At first, especially during prototyping, this can be easy to miss, because nothing looks obviously broken. Coverage stays high. The system works. But over time, certain parts of the codebase become strangely resistant to cleanup.

Tests that protect the wrong behaviour create drag against coherence. The harder you push for coverage, the more deeply you can lock in the shape you happened to start with.

This isn't a criticism of tests. Tests are necessary. But tests primarily validate behaviour, while many of the emerging failure modes in agent-generated systems, especially as they grow, are failures of *shape*.

> A load-bearing poster.
>
> "Uh, I wouldn't take it down if I were you. It's a load-bearing poster."

Fitness functions protect the shape of the system

Unit tests ask whether the code still does what it did before. Fitness functions ask a different question: is the system still shaped the way we want?

Most codebases already have a few simple versions of this. Linters, type checks, and import-boundary rules all encode some idea of acceptable shape, at the micro level.

But in agentically-maintained codebases, we need an additional kind of macro fitness function. It has a couple of important jobs:

- catch bad agent choices at the moment of work
- redirect agents with the right amount of friction for the violation

At Forestwalk, for example, we have a fitness check that prevents tool handlers from calling back into our AI services. Tool handlers run inside the agent's own loop, so any handler that kicks off more inference can stack model calls inside model calls and freeze a live meeting. Agents sometimes accidentally introduce this pattern, so our fitness check flags it and ensures it's not committed.

Other fitness tests are ratchets that apply pressure to measurable properties – heuristics for shape like file size, bundle growth, prompt size, import fanout, and dependency count.

For these, the ceiling starts at the current value (e.g. 837kb) and flags to agents and humans if a PR would unduly increase it. For instance, when an agent adds an unnecessary new dependency that blows up our frontend bundle, the check fails.

The important part isn't just the limit itself. It's the “just in time” agent coaching.

At first, we found agents would sometimes over-optimize against whatever metric the checks exposed, with great resourcefulness and in ways we did not want. But refining the flags to give common sense guidance helped steer the agents into considering tradeoffs correctly.

For example, an agent might see this just in time:

```
□□□□□□□ Failed Checks: 1 □□□□□□□

FAIL  src/frontendBundleBudget.test.ts > Frontend bundle size budget
AssertionError: Frontend bundle grew +29 KiB (700 -> 729 KiB; fails a

This ratchet is review pressure, not a mandate to shrink the number a

Diagnose first:
- Is the growth accidental (dependency, duplicate/dead code, broad im
- Does the added JS unlock user-visible value?
- Can cleanup reduce it without hurting UX, accessibility, tests, or

Quality-preserving fixes:
```

```
- remove duplicated rendering paths or dead production code
- move repeated styling or static configuration out of lazy-loaded Ja
- lazy-load rarely used flows instead of loading them on the main pat
(more examples here)

Do not fix this by:
- omitting requested scope or deferring needed UI work
- removing aria-label/title text or visible affordances
- deleting helpful empty/error-state copy
(more examples here)

For intentional growth, (procedure here) and explain why in the PR.
```

If you simply tell agents to "reduce bundle size," they will sometimes do naive things like:

- delete empty-state copy
- replace styled controls with browser-native ones
- inline dependency code to avoid import costs
- remove accessibility affordances

Instead, the idea is to not just explain what a given fitness test is protecting, but also explain what is *not allowed* to be sacrificed in the process, and do so at the moment the agent is working.

In contrast, the same guidance in `CLAUDE.md` is often skimmed past or applied inconsistently by agents. A just-in-time check, on the other hand, creates desired friction between the agent and a finished task. Where humans get tired and occasionally creative about routing around friction like this, agents are quite willing to schlep through feedback and loop until the check is passing.

> A master bedroom.
>
> "Come on in, it's your master bedroom!"

What the checks can't see

Fitness functions are good at catching things precise enough to encode.

That is useful, but coherence is not always that crisp. Some drift is obvious only in relation to the rest of the codebase: this new thing is too close to an existing thing, this test is protecting a pattern we no longer want, this change is making the next change harder.

So in addition, we use a few non-deterministic checks on top of the deterministic ones.

One runs before a PR is opened. It scans the proposed change against the existing codebase, and asks whether the agent has introduced or expanded a competing pattern. When it finds one, it points to the existing pattern, explains why the new one looks redundant, and asks the agent to either extend the original or justify why this case is genuinely different.

Most of the time, the agent course-corrects quickly.

We run a similar check again at the PR review level, where it has more context and can catch subtler pattern drift.

Another check runs nightly and looks for tests that have fossilized patterns we have since decided against. Its mission is to find tests that protect old implementation choices that, if left alone, would teach future agents to preserve patterns we no longer want.

These checks are not magic. They miss things. They can be noisy. They still need human judgment around them.

But the goal is not to eliminate judgment. The goal is to move more human judgment into places where it can be highest leverage.

Making coherence the easy path

Fitness ratchets are not the whole answer to maintaining coherence in agent-generated codebases.

Fitness functions are one layer. Pattern drift checks are another. Product verification, code review, evals, documentation, repository structure, and the actual taste of the humans steering the system all matter too.

However, it's worth investing regularly in the early layers: the ones closest to the moment an agent is making an implementation choice.

That is where we have found these types of fitness functions useful. They do not solve coherence by themselves, but they add friction in the right places. Bad patterns get interrupted earlier. Good patterns have a better chance of becoming precedent.

The same feedback loop that spreads bad patterns can be used to spread good ones.

So there’s hope. When you see this pattern work, it soothes some of the worry we all have about agents causing chaos. They’re a lot more useful when the codebase gives them an obvious next move. The right pattern is easier to find. The weird fourth version of the thing has a harder time looking reasonable.

It's not perfect. Keeping a growing codebase coherent still requires human judgment, architectural intuition, and social coordination. The system still needs to be built.

But that system, itself, increasingly feels like the work.

If the codebase trains the next agent, then the job is not just to write better code.

It is to build the environment where better code is the obvious thing to write.